

Datasets empleados en los papers e información adicional

Laura Cabaleiro

Diciembre 2025

Datasets de Papers

1. G. I. Webb et al. – "Supervised Learning from Data Streams: An Overview and Update" (ACM Computing Surveys, 2024)

Se mencionan datasets populares que se han utilizado en la investigación sobre el aprendizaje de flujos de datos, pero se hacen algunas aclaraciones importantes sobre su naturaleza:

- **Forest Coverttype:** dataset clásico utilizado en investigaciones de minería de datos. Sin embargo, el artículo señala que este dataset *no está diseñado como un flujo de datos*. Dataset estático que se ha *transformado artificialmente* en un flujo de datos para ciertos experimentos.
URL: <https://archive.ics.uci.edu/ml/datasets/Coverttype>
- **Poker Hand:** dataset que no es un flujo de datos real, convertido en un flujo artificialmente. Usado para probar algoritmos de clasificación, y fuera del contexto de flujos de datos.
URL: <https://archive.ics.uci.edu/ml/datasets/Poker+Hand>
- **Electricity Dataset:** utilizado como un benchmark en flujos de datos, particularmente para el análisis de dependencia temporal en los datos. Aunque se menciona en el artículo como un dataset de referencia, también se critica por no ser un flujo de datos real, ya que no está diseñado específicamente para estudiar el concepto de "drift" en flujos de datos.
URL: <https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014>

2. A. Halstead et al. – "Recurrent Concept Drifts on Data Streams" (IJCAI, 2024)

Repositorios de Código y Software:

- **MOA (Massive Online Analysis):** El artículo menciona que muchos de los métodos discutidos tienen implementaciones disponibles en MOA, como LEARN++.NSE, PCCF y ProChange. Las implementaciones de CPF, ECPF, MDP y PEARL también están disponibles como repositorios separados en MOA.
- **scikit-multiflow:** Repositorios basados en scikit-multiflow, como scikit-ika, que contiene implementaciones de PEARL y Nacre.
- **River:** El artículo menciona FALL (una plataforma modular de aprendizaje basada en River) que permite el uso de clasificadores y detectores de drift de River. También implementa los mecanismos de selección y evaluación de contexto C-F1 utilizados en métodos como FiCSUM.
- **Repositorios en GitHub:** Se mencionan forks en GitHub de frameworks populares de aprendizaje automático para flujos de datos como MOA, scikit-multiflow y River, con implementación de los métodos revisados en el artículo.

Datasets:

- **Datasets sintéticos:** Mencionan generadores de datos sintéticos como SEA, Hyperplane, Agrawal, Random Tree y LED, disponibles en MOA (Massive Online Analysis).
URL: <https://moa.cms.waikato.ac.nz/>
- **Datasets reales:**
 - **Electricity Load Diagrams:** Dataset utilizado en la literatura de aprendizaje supervisado en flujos de datos, especialmente como benchmark para el análisis de dependencia temporal y drifts recurrentes.
URL: <https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014>
 - **PEMS-SF (Traffic Sensor Dataset):** Dataset de sensores de tráfico urbano utilizado en estudios de flujos de datos con cambios temporales y patrones recurrentes.
URL: <https://archive.ics.uci.edu/ml/datasets/PEMS-SF>
 - **Mosquito Wingbeat Audio Datasets (Aedes y Culex):** Conjuntos de datos de señales acústicas del batido de alas de mosquitos de los géneros *Aedes* y *Culex*, utilizados para tareas de clasificación y análisis de patrones recurrentes influenciados por condiciones ambientales.
URL: <https://www.kaggle.com/datasets/potamitis/wingbeats>

Fórmulas:

El artículo aborda métodos estadísticos utilizados para detectar drifts recurrentes, y cómo se manejan los conceptos recurrentes:

- **Jensen-Shannon Divergence:** Se utiliza en métodos como ESCR para medir la divergencia entre distribuciones de puntuaciones de confianza de clasificadores, lo que permite detectar la aparición de drifts recurrentes.
- **Cumulative Accuracy Gain (CAG):** Se menciona como una métrica para evaluar métodos en flujos de datos con drifts recurrentes, comparando el rendimiento de un método propuesto frente a un baseline a lo largo del tiempo.

Sea un flujo de datos y dos clasificadores: el método propuesto A y un baseline B . Sea $m \in \mathbb{N}$ la *frecuencia de muestreo*, es decir, cada cuántas instancias se calcula la métrica. Definimos los instantes de evaluación como $t \in \{1, \dots, T\}$, donde el instante t corresponde al bloque de instancias $\{(t-1)m+1, \dots, tm\}$.

Denotamos por $\text{Acc}_A(t)$ y $\text{Acc}_B(t)$ la precisión (accuracy) calculada en el bloque t para A y B , respectivamente. La CAG acumulada hasta T evaluaciones se define como:

$$\text{CAG}(T) = \sum_{t=1}^T (\text{Acc}_A(t) - \text{Acc}_B(t)). \quad (1)$$

Esta métrica refleja una suma acumulada de las diferencias de precisión frente a un baseline, evaluada a una frecuencia de muestreo dada [?].

Además, se emplean técnicas de agrupación no supervisada (como el algoritmo k-Means) para crear clusters de conceptos y facilitar la detección de drifts recurrentes.

3. S. Hinder et al. – “Data streams classification using deep learning under different concept drifts” (Journal of Logic and Computation, 2023)

Datasets:

- **UCR Repository:** Se utilizan 29 datasets de time series provenientes del UCR repository, los cuales se simulan como flujos de datos. Estos datasets cubren diferentes dominios y características, y se usan para evaluar el rendimiento de los modelos de aprendizaje profundo. Ejemplos de estos datasets incluyen TwoPatterns, CinCECGtorso, Pendigits, y ElectricDevices. Se mencionan con detalles como el número de instancias, longitud de la serie temporal y el número de clases.
URL: https://www.cs.ucr.edu/~eamonn/time_series_data/
- **Concept Drift Datasets:** Se generan 15 datasets sintéticos diseñados para simular distintos tipos de concept drift. Estos incluyen cambios abruptos, graduales e incrementales en las distribuciones de datos, utilizando generadores como RBF, LED, RandomTree, Agrawal, y SEA. Estos se implementan usando la librería River.
URL: <https://riverml.xyz/>

Repositorios:

- **ADLStream Framework:** El código del ADLStream Python library (framework utilizado en el estudio para la clasificación de flujos de datos) está disponible en este repositorio de GitHub: <https://github.com/pedrolarben/ADLStream>
- **Deep Learning Online Classification:** La implementación de los experimentos de clasificación de datos en tiempo real, utilizando el framework de ADLStream, también está disponible en este otro repositorio: <https://github.com/pedrolarben/deep-learning-online-classification>

Fórmulas. El artículo utiliza la precisión prequential y el Kappa Statistic como métricas para evaluar el rendimiento de los modelos en flujos de datos.

Precisión prequential (test-then-train) con factor de decaimiento. En evaluación prequential, cada instancia k se utiliza primero para realizar una predicción $\hat{y}_k$ y, una vez conocida la etiqueta real, para actualizar el modelo [?]. Sea y_k la etiqueta real y $\hat{y}_k$ la predicción correspondiente en el instante k .

La precisión prequential con decaimiento exponencial $\alpha \in (0, 1]$ se define como:

$$P_\alpha(i) = \frac{\sum_{k=1}^i \alpha^{i-k} \mathbb{I}(\hat{y}_k = y_k)}{\sum_{k=1}^i \alpha^{i-k}}. \quad (2)$$

De forma equivalente, esta métrica puede expresarse en términos de una función de pérdida $L(y_k, \hat{y}_k)$, manteniendo la misma ponderación exponencial:

$$P_\alpha(i) = \frac{\sum_{k=1}^i \alpha^{i-k} L(y_k, \hat{y}_k)}{\sum_{k=1}^i \alpha^{i-k}} = \frac{S_\alpha(i)}{B_\alpha(i)}. \quad (3)$$

Kappa Statistic. El estadístico Kappa se define como:

$$\kappa = \frac{p_0 - p_c}{1 - p_c}, \quad (4)$$

donde p_0 es la precisión prequential observada y p_c es la probabilidad de acuerdo esperado por azar.

4. M. Mena-Torres et al. – “A survey on machine learning for recurring concept drifting data streams” (2023)

No se mencionan datasets, fórmulas o código específico. Se incluye una descripción de métodos de Drift Detection:

- **ADWIN y ADWIN2:** Se utilizan para detectar concept drift en el flujo de datos, basándose en la diferencia de medias entre dos ventanas de datos.

- DDM (Drift Detection Method): Detecta el drift observando la tasa de error en el clasificador.
- RDDM: Un método reactivo para detectar drift, ajustando el umbral cuando se detectan cambios.
- HDDM: Utiliza desigualdad de Hoeffding para detectar cambios abruptos y graduales en los flujos de datos.

5. N. Lu et al. – “One or two things we know about concept drift—a survey on unsupervised drift detection” (Frontiers in Artificial Intelligence, 2024)

Datasets:

- **Uniform Dataset:** Este dataset se genera de manera uniforme dentro de un cuadrado unitario, con el drift introducido por un desplazamiento a lo largo de la diagonal.
URL: <https://github.com/FabianHinder/One-or-Two-Things-We-Know-about-Concept-Drift>
- **Gaussian Dataset:** Los datos son generados a partir de una distribución normal con características correlacionadas, y el drift cambia la señal de correlación entre las características.
URL: <https://github.com/FabianHinder/One-or-Two-Things-We-Know-about-Concept-Drift>
- **Two Overlapping Dataset:** Los datos provienen de dos cuadrados uniformes superpuestos, y el drift se genera mediante una rotación de 90 grados.
URL: <https://github.com/FabianHinder/One-or-Two-Things-We-Know-about-Concept-Drift>

Fórmulas:

- **Kolmogorov-Smirnov Test:** Se utiliza para medir la discrepancia máxima entre dos muestras. La fórmula para la estadística del test es:

$$\hat{d}(S^-(t), S^+(t)) = \sup_x |F_{S^+(t)}(x) - F_{S^-(t)}(x)|$$

donde F es la función de distribución empírica acumulada (CDF) de cada muestra.

- **MMD (Maximum Mean Discrepancy):** Utiliza kernels para medir la discrepancia entre dos distribuciones de muestras P y Q , y se expresa como:

$$\text{MMD}(P, Q) = \max_{f \in \mathcal{H}} |\mathbb{E}_{X \sim P}[f(X)] - \mathbb{E}_{X \sim Q}[f(X)]|$$

donde $\mathcal{H}$ es el espacio de funciones de un kernel elegido, y f es una función de ese espacio.